Лабораторная работа 4

Self-Supervised Representation Learning

Contrastive learning, invariances, collapse diagnostics и label efficiency

Курс «Нейронные сети»

Редакция 4 от 8 октября 2026 г.

1 Введение

Разметка данных стоит дорого, а неразмеченных изображений обычно много. В курсе обучающая часть CIFAR-10 содержит 43 000 изображений; 1% из них — всего 430 изображений, по 43 на класс. Обучить на них глубокую сеть с нуля трудно.

Обучение без разметки (self-supervised learning) использует все изображения и не требует меток. В методе SimCLR [8] из каждого изображения делают два случайно искажённых вида (views): обрезают, меняют цвета, отражают. Сеть учится узнавать виды одного изображения среди видов других изображений — это контрастное обучение (contrastive learning). Так строится представление (representation) — вектор признаков, по которому затем обучают простой классификатор на немногих метках.

Человек узнаёт кошку и на обрезанном, и на чёрно-белом снимке. SimCLR учит сеть тому же: не замечать изменений, которые вносят аугментации — случайные преобразования изображений. Поэтому набор аугментаций решает, что представление сохранит, а что потеряет. Если класс определяется цветом, а аугментации меняют цвет, представление окажется бесполезным.

Все эксперименты выполняются в проекте из ЛР0: у каждого запуска есть `EXP_ID`, `RUN_ID`, паспорт и итоговая конфигурация. Числа в отчёте сопровождаются ссылками на `RUN_ID`.

2 Цель работы

Цель работы — построить и исследовать self-supervised representation learning pipeline: обучить SimCLR на изображениях CIFAR-10 без меток и сравнить его с базовыми моделями. Требуется также измерить вклад каждой аугментации и экономию меток.

Работа учит различать четыре свойства, которые не следуют одно из другого:

1. функция потерь предобучения уменьшилась;
2. представление не выродилось, то есть нет коллапса;
3. классы линейно разделимы в пространстве представлений;
4. представление помогает, когда меток мало.

Какие инвариантности задаёт набор аугментаций SimCLR? Как проверить, что представление полезно, если метки при обучении не использовались?

3 Что нужно знать заранее

Если какой-то пункт незнаком, восполните пробел по указанному источнику.

- Свёрточные сети и архитектура ResNet — [1, разд. 5.2, 5.4], [2, разд. 9.1–9.3].
- Пополнение набора данных (аугментации) — [2, разд. 7.4]; преобразования в torchvision — [3].
- Перекрёстная энтропия, логистическая регрессия и метод ближайших соседей — [1, разд. 2.3], [2, разд. 5.7].
- Что такое представление и зачем нужно предобучение — [2, разд. 15.1, 15.2, 15.4].
- Ковариационная матрица, след, собственные значения — [2, разд. 2.7, 2.10, 2.12, 3.8].
- Нормализация по мини-батчам и режимы `train` и `eval` модели — [1, разд. 4.3], [6].
- Проект для экспериментов из ЛР0.

4 Результаты обучения

После выполнения работы вы сможете:

1. объяснять, как SimCLR учится без меток и какую роль играют отрицательные примеры и температура;
2. реализовывать и проверять контрастную функцию потерь NT-Xent;
3. объяснять, какую инвариантность задаёт каждая аугментация, и измерять её вклад;
4. оценивать представление линейным probeом, методом ближайших соседей и диагностикой коллапса и понимать, что измеряет каждый способ;
5. строить зависимость качества от доли разметки и делать вывод по нескольким seeds с указанием источника случайности.

5 Модели и обозначения

5.1 Два вида одного изображения

Аугментация меняет изображение, но не его смысл: обрезанная или обесцвеченная фотография кошки — всё ещё кошка. SimCLR дважды применяет к изображению случайные преобразования и получает два его вида:

$$\tilde{x}_i^{(1)} = t_1(x_i), \qquad \tilde{x}_i^{(2)} = t_2(x_i), \qquad t_1, t_2 \sim \mathcal{T} \text{ независимо}, \tag{4.1}$$

где x_i — изображение, t_1 и t_2 — независимые случайные преобразования из набора аугментаций $\mathcal{T}$. Каждое преобразование состоит из m операций:

$$t = o_m \circ \cdots \circ o_2 \circ o_1, \tag{4.2}$$

где операция o_j, например обрезка или изменение цвета, применяется с вероятностью p_j. Её сила задаётся в конфигурации. Набор $\mathcal{T}_{-j}$ — это $\mathcal{T}$ без операции o_j.

Сеть учат давать двум видам близкие представления. Поэтому всё, что меняют аугментации, представление учится не замечать: оно становится *инвариантным* к этим изменениям. Изменение цвета даёт инвариантность к цвету, обрезка — к положению и масштабу объекта. Инвариантность полезна, только если изменяемый признак не нужен задаче [9, 10].

Подробнее: пополнение набора данных — [2, разд. 7.4]; преобразования torchvision — [3].

5.2 Кодировщик и проекционная голова

Кодировщик (encoder) f_θ переводит вид в представление h. Над ним стоит проекционная голова (projection head) g_ϕ — небольшая сеть, по выходу z которой считается функция потерь:

$$h = f_\theta(\tilde{x}) \in \mathbb{R}^{d_h}, \qquad z = g_\phi(h) = W_2\,\mathrm{ReLU}(W_1 h) \in \mathbb{R}^{d_z}, \tag{4.3}$$

где $W_1 \in \mathbb{R}^{d_p \times d_h}$ и $W_2 \in \mathbb{R}^{d_z \times d_p}$ — матрицы головы без смещений. Кодировщик — ResNet-18 [11] в варианте для CIFAR: первая свёртка 3×3, без max-pooling. Размерности: $d_h = 512$, по умолчанию $d_p = 512$ и $d_z = 128$.

После обучения голову отбрасывают, а качество оценивают по h [8]. Голова учится не замечать аугментаций. Поэтому она может отбросить сведения, полезные другим задачам, например цвет объекта. Сходство двух проекций — косинус угла между ними, от -1 до 1:

$$\hat{z} = \frac{z}{\|z\|_2}, \qquad \mathrm{sim}(z_i, z_k) = \hat{z}_i^\top \hat{z}_k. \tag{4.4}$$

Подробнее: ResNet — [1, разд. 5.4]; архитектуры torchvision — [4].

5.3 Контрастная функция потерь NT-Xent

Обучение SimCLR устроено как угадывание. Batch из N изображений даёт $2N$ видов. Для вида i нужно найти среди остальных $2N - 1$ видов его пару $j(i)$ — второй вид того же изображения. Это положительная пара (positive pair), а остальные $2N - 2$ вида — отрицательные примеры (negatives). Угадывание оформлено как классификация с перекрёстной энтропией — функция потерь NT-Xent [8, 12]:

$$\ell_i = -\ln \frac{\exp\bigl(s_{i,j(i)}/\tau\bigr)}{\sum_{k \neq i} \exp\bigl(s_{ik}/\tau\bigr)}, \qquad \mathcal{L} = \frac{1}{2N} \sum_{i=1}^{2N} \ell_i, \tag{4.5}$$

где $s_{ik} = \mathrm{sim}(z_i, z_k)$, $\tau > 0$ — температура, ℓ_i — потери вида i. Сходство вида с самим собой исключено, поэтому в знаменателе $2N - 1$ слагаемых.

Если все сходства одинаковы, $\ell_i = \ln(2N - 1)$ — уровень случайного угадывания, как $\ln K$ в ЛР1. При $N = 256$ он равен $\ln 511 \approx 6{,}24$. Разность $\ln(2N - 1) - \mathcal{L}$ оценивает снизу взаимную информацию двух видов [12]. В SimCLR оценка приближённая, так как отрицательные примеры приходят парами, и никогда не превышает $\ln(2N - 1)$.

Достижимые значения функции потерь зависят и от τ. Если пары совпадают, а отрицательные примеры перпендикулярны, то $\ell_i = \ln\bigl(1 + (2N - 2)e^{-1/\tau}\bigr)$. При $N = 256$ это около 4,25 для $\tau = 0{,}5$ и около 0,02 для $\tau = 0{,}1$. Поэтому функции потерь сравнимы только при одинаковых N и τ.

Подробнее: перекрёстная энтропия — [1, разд. 2.3]; отрицательные примеры в word2vec — [1, разд. 7.2]; контрастные функции потерь — [7].

5.4 Как температура распределяет отталкивание

Роль τ видна из градиента потерь вида i по нормированной проекции, без учёта производной нормировки:

$$\frac{\partial \ell_i}{\partial \hat{z}_i} = \frac{1}{\tau}\Big[\big(p_{i,j(i)} - 1\big)\,\hat{z}_{j(i)} + \sum_{k \neq i,\, j(i)} p_{ik}\,\hat{z}_k\Big], \qquad p_{ik} = \frac{\exp(s_{ik}/\tau)}{\sum_{k' \neq i} \exp(s_{ik'}/\tau)}, \tag{4.6}$$

где p_{ik} — доля кандидата k в знаменателе (4.5), сумма p_{ik} по $k \neq i$ равна единице. Шаг градиентного спуска идёт против градиента. Поэтому вид притягивается к своей паре с весом $1 - p_{i,j(i)}$. От отрицательного примера k он отталкивается с весом p_{ik}, сильнее всего — от самых похожих [13].

Температура задаёт, насколько сосредоточены веса. Пусть сходства двух отрицательных примеров равны 0,9 и 0,5. Отношение их весов $e^{0{,}4/\tau}$ равно примерно 55 при $\tau = 0{,}1$ и 1,5 при $\tau = 1$. Множитель $1/\tau$ меняет и масштаб градиента, поэтому τ связана со скоростью обучения. Отталкивание распределяет проекции по сфере равномерно [14]. Без него минимум потерь достигался бы и тогда, когда сеть выдаёт один вектор для всех изображений.

5.5 Коллапс представлений и эффективный ранг

Коллапс (collapse) — вырождение представления. При полном коллапсе все изображения получают один вектор. При коллапсе по размерностям (dimensional collapse) векторы занимают лишь несколько из d направлений [15]. Оба случая видны по ковариации представлений $h_1, \ldots, h_n \in \mathbb{R}^d$ фиксированной диагностической выборки, где $d = d_h$ или d_z:

$$\bar{h} = \frac{1}{n}\sum_{i=1}^{n} h_i, \qquad C = \frac{1}{n-1}\sum_{i=1}^{n} (h_i - \bar{h})(h_i - \bar{h})^{\top}. \tag{4.7}$$

Собственные значения $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$ матрицы C — дисперсии вдоль главных направлений, их сумма $\operatorname{tr} C$ — масштаб разброса. Эффективный ранг (effective rank) показывает, по скольким направлениям распределена дисперсия [16]:

$$q_j = \frac{\lambda_j}{\operatorname{tr} C}, \qquad r_{\text{eff}} = \exp\Big(-\sum_{j=1}^{d} q_j \ln q_j\Big), \qquad 1 \leq r_{\text{eff}} \leq d, \tag{4.8}$$

где q_j — доля дисперсии направления j, $0 \ln 0 = 0$. Если 97% дисперсии приходится на одно направление и по 1% на три других, то $r_{\text{eff}} \approx 1{,}18$, а не 4.

Величина (4.8) определена только при $\operatorname{tr} C > 0$, а малая константа в знаменателе дала бы произвольное число. Поэтому принята конвенция:

- если $\operatorname{tr} C \leq \varepsilon_c\, d\, \overline{\|h\|^2}$ при $\varepsilon_c = 10^{-6}$, где $\overline{\|h\|^2}$ — средний квадрат нормы, вместо r_{eff} пишется `n/a` и `collapse_scale = true`;
- r_{eff} всегда приводится вместе с $\operatorname{tr} C$ и средней нормой;
- малое r_{eff}/d при нормальном масштабе — признак коллапса по размерностям; порог по умолчанию 0,2 диагностический.

У нормированных проекций полный коллапс виден по длине среднего вектора $\bar{\hat{z}} = \frac{1}{n}\sum_i \hat{z}_i$:

$$\operatorname{tr} C_{\hat{z}} = \frac{n}{n-1}\big(1 - \|\bar{\hat{z}}\|_2^2\big). \tag{4.9}$$

Если все проекции совпали, $\|\bar{\hat{z}}\| = 1$. У равномерно разбросанных направлений $\|\bar{\hat{z}}\| \approx 1/\sqrt{n}$. Связь ранга с качеством классификации установлена лишь эмпирически [17].

Подробнее: ковариация, след и спектральное разложение — [2, разд. 2.7, 2.10, 2.12, 3.8].

5.6 Оценка представлений

Полезность представления проверяют на задаче с метками тремя способами.

Linear probe. Кодировщик замораживают и обучают поверх него только линейный классификатор:

$$\hat{y} = \mathrm{softmax}(Wh + b), \qquad W \in \mathbb{R}^{K \times d_h},\ b \in \mathbb{R}^K, \tag{4.10}$$

где K — число классов. Заморозить нужно и параметры, и буферы BatchNorm — накопленные средние и дисперсии. Для этого кодировщик переводят в режим `eval` [6]. Признаки h вычисляют один раз без аугментации, и probe становится логистической регрессией. Её регуляризацию выбирают по `val`.

Метод ближайших соседей (kNN). Класс определяется взвешенным голосованием k самых похожих изображений `train` [18]:

$$\hat{y}(x) = \arg\max_{c} \sum_{i \in \mathcal{N}_k(x)} \mathbb{1}[y_i = c]\, \exp\bigl(\mathrm{sim}(h(x), h_i)/\tau_{\mathrm{kNN}}\bigr), \tag{4.11}$$

где $\mathcal{N}_k(x)$ — эти изображения, y_i — их метки. По умолчанию $k = 200$, $\tau_{\mathrm{kNN}} = 0{,}07$. Метод ничего не обучает: он проверяет, лежат ли рядом изображения одного класса. probe проверяет, разделимы ли классы линейно.

Выигрыш при малой разметке. Пусть размечена доля r изображений `train`. Выигрыш предобучения — разность точностей probe на `test` с кодировщиком SimCLR и со случайным [8]:

$$G(r) = M_{\mathrm{SSL}}(r) - M_{\mathrm{rand}}(r). \tag{4.12}$$

Парные разности. Главный вывод проверяется на трёх общих seeds, как в ЛР1:

$$\Delta_s = M(\mathrm{A}, s) - M(\mathrm{B}, s), \qquad \bar{\Delta} = \frac{1}{3}\sum_{s=1}^{3} \Delta_s, \qquad s_\Delta = \sqrt{\frac{1}{2}\sum_{s=1}^{3} (\Delta_s - \bar{\Delta})^2}, \tag{4.13}$$

где $M(\mathrm{A}, s)$ — метрика конфигурации A при seed s. Источник случайности, который меняется вместе с seed, называется явно: от него зависит, какой разброс измерен.

Подробнее: логистическая регрессия и метод ближайших соседей — [2, разд. 5.7]; перенос представлений — [2, разд. 15.2].

5.7 Сводка обозначений

Обозначение	Смысл
x_i, $\tilde{x}_i^{(1)}$, $\tilde{x}_i^{(2)}$; $\mathcal{T}$, $\mathcal{T}_{-j}$, o_j, p_j, m	изображение и его виды; набор аугментаций, набор без o_j, операция, её вероятность, число операций
f_θ, g_ϕ, h, z, $\hat{z}$; d_h, d_p, d_z	кодировщик, голова, представление, проекция, нормированная проекция; размерности
N, $j(i)$, s_{ik}, τ, ℓ_i, $\mathcal{L}$, p_{ik}	число изображений в batch, номер пары вида i, сходство, температура, потери вида и batch, веса кандидатов
n, C, λ_j, q_j, r_{eff}, ε_c, $\bar{\hat{z}}$	размер диагностической выборки, ковариация, собственные значения, их доли, эффективный ранг, порог масштаба, средняя нормированная проекция
K, k, τ_{kNN}; r, M, $G(r)$	число классов, соседей и температура в kNN; доля разметки, точность, выигрыш предобучения
Δ_s, $\bar{\Delta}$, s_Δ	парные разности, их среднее и стандартное отклонение

6 Данные и оборудование

Основной набор — CIFAR-10 [19]: 60 000 цветных изображений 32×32, 10 классов. Разбиение взято из ЛР0: `train` — 43 000 изображений, `val` — 5 000, `cal` — 2 000, `test` — 10 000. Часть `cal` нужна в ЛР8. Эта работа её не читает, и проект отказывает в чтении.

Правила обучения без разметки:

- SimCLR обучается на изображениях `train` без меток;
- изображения `val` и `test` в предобучении не используются никак, даже для выбора аугментаций и ранней остановки;
- метки `train` нужны только кодировщику с учителем, probey и kNN; всё выбирается по `val`, а `test` нужен для итоговых чисел.

Кадры лиц для дополнительного задания 3 используются по «Протоколу данных курса».

Оборудование. Нужна видеокарта: подойдут бесплатные Google Colab или Kaggle. Сеансы там ограничены, поэтому продолжайте обучение с контрольной точки механизмом ЛР0. Без видеокарты используйте уменьшенный вариант. Возьмите компактную ResNet шириной 32 канала и 20 эпох вместо 100. В E2 уберите по очереди две операции, а не все три.

7 Индивидуальный профиль

Как и в ЛР1, параметры работы вычисляются функцией `lab_pick` из хеша номера зачётки вместе с номером работы. Добавьте в генератор профиля три поля ЛР4:

```
def lab_pick(student_id: str, lab: int, a: int, b: int, options: list):
    hk = hashlib.sha256(f"{student_id}:lab{lab}".encode("utf-8")).hexdigest()
    return options[int(hk[a:b], 16) % len(options)]

profile["lab4_tau"] = lab_pick(sid, 4, 0, 2, [0.1, 0.2, 0.5])
profile["lab4_third_aug"] = lab_pick(sid, 4, 2, 4,
```

```
                                        ["grayscale", "flip", "blur"])
profile["lab4_label_fractions"] = lab_pick(sid, 4, 4, 6, [
    [0.01, 0.05, 0.25, 1.0], [0.01, 0.10, 0.25, 1.0], [0.02, 0.05, 0.25, 1.0]])
```

Здесь `sid` — номер зачётки, `profile` — словарь профиля из ЛР0. Профиль задаёт температуру τ, третью операцию базового набора аугментаций и доли разметки r для E3. Первые две операции у всех одинаковы: обрезка с масштабированием и изменение цвета. Наименьшая доля $r_{\min}$ — главная точка E3: при $r_{\min} = 0{,}01$ размечено 430 изображений. Пример: для номера `20250137` получается $\tau = 0{,}1$, третья операция — отражение, доли 0,01, 0,10, 0,25 и 1,0.

8 Задания

8.1 Задание 1. Два вида одного изображения: набор аугментаций

Что сделать.

1. Реализуйте набор данных, который возвращает два вида изображения по формуле (4.1):

```
view1, view2, meta = dataset[index]    # meta: image index
```

2. Реализуйте на основе torchvision [3] пять операций: обрезку с масштабированием, отражение, изменение яркости, контраста, насыщенности и оттенка, перевод в оттенки серого и размытие. Порядок операций (4.2), вероятности p_j и сила задаются в конфигурации. Значения примера близки к настройкам SimCLR [8]:

```
augmentations:   # 3rd op: flip, grayscale {p: 0.2} or blur {p: 0.5}
  - {name: random_resized_crop, p: 1.0, scale: [0.08, 1.0]}
  - {name: color_jitter, p: 0.8, brightness: 0.4, contrast: 0.4,
     saturation: 0.4, hue: 0.1}
  - {name: flip, p: 0.5}
```

3. Присваивайте каждому набору идентификатор — хеш параметров. Наборы $\mathcal{T}_{-j}$ для E2 порождайте из базового программно.
4. Отделите генератор аугментаций от генераторов разбиения и инициализации. Проверьте, что виды не совпадают из-за неверного seed, в том числе в процессах-загрузчиках (workers) [5].
5. Добавьте режим без аугментации: перевод в тензор и нормализация по статистикам `train`. В нём вычисляются признаки в задании 3.

Что должно получиться. Любой набор аугментаций, включая варианты для E2, создаётся конфигурацией без правки кода.

Как проверить себя. Сохраните сетку из восьми изображений и их видов: виды должны оставаться узнаваемыми. Тест 1 проходит.

8.2 Задание 2. Контрастная функция потерь NT-Xent

Что сделать.

1. Реализуйте функцию (4.5) сами, без готовых функций библиотек обучения без разметки. Вход — проекции двух видов, две матрицы $N \times d_z$.
2. Склейте их в матрицу $2N \times d_z$, нормируйте по (4.4) и вычислите матрицу сходств $2N \times 2N$. Пара вида i — вид $(i + N) \bmod 2N$ при нумерации с нуля.
3. Поставьте на диагональ $-\infty$, чтобы исключить сходство вида с собой. В знаменателе останется $2N - 1$ слагаемых.
4. Делите сходства на τ до `logsumexp` и не вычисляйте экспоненты напрямую. При $\tau = 0{,}01$ число $e^{100} \approx 2{,}7 \cdot 10^{43}$ не помещается в `float32`, предел которого около $3{,}4 \cdot 10^{38}$.
5. Возвращайте потери, число отрицательных примеров $2N - 2$ и средние сходства положительных и отрицательных пар. Числа $2N - 2$ и $\ln(2N - 1)$ запишите в паспорт запуска.
6. Соберите обучение SimCLR: виды → кодировщик → голова → функция потерь. Используйте цикл обучения и контрольные точки ЛР0.

Что должно получиться. SimCLR обучается одной командой по конфигурации, а функция потерь проверена на примерах с известным ответом.

Как проверить себя. Тесты 2 и 3 проходят. В начале обучения функция потерь близка к $\ln(2N - 1)$ или ниже. Если она заметно выше, проверьте номера пар.

8.3 Задание 3. Оценка представлений: Linear probe, kNN, эффективный ранг

Что сделать.

1. Вычисляйте признаки h и $\hat{z}$ для `train`, `val` и `test` в режиме `eval` без аугментации. Сохраняйте их вместе с `RUN_ID` кодировщика.
2. Реализуйте Linear probe (4.10) на сохранённых признаках: регуляризация — по `val`, точность — на `test`.
3. Реализуйте kNN (4.11) с k и τ_{kNN} из конфигурации. Соседей ищите только в `train`.
4. Реализуйте диагностику h и $\hat{z}$ на 5 000 фиксированных изображениях `train`, индексы которых записаны в манифест. Вычисляйте среднюю норму, $\operatorname{tr} C$, спектр C, r_{eff} и r_{eff}/d по конвенции раздела 5.5 и $\|\bar{\hat{z}}\|$. Добавьте средние сходства положительных пар, построенных с фиксированным seed, и случайных пар.
5. Сохраняйте сводку в `summary.json` кодировщика:

```
"representation": {
  "h": {"linear_probe": ..., "knn": ..., "trace_cov": ...,
        "effective_rank": ..., "collapse_scale": false},
  "z_hat": {"mean_zhat_norm": ..., "positive_similarity": ...}}
```

6. Реализуйте подвыборки меток $S_s(r)$ для Е3 — для каждого seed s и доли r, стратифицированные по классам. Подвыборки одного seed вложены друг в друга. Индексы сохраняйте в манифест ЛР4.

Что должно получиться. По `RUN_ID` кодировщика одна команда выдаёт сводку: точности probe и kNN, масштаб, эффективный ранг и признак коллапса.

Что добавляется в общий проект. Модули `src/ssl/` (`views.py`, `augmentations.py`, `losses.py`, `simclr.py`), `src/evaluation/` (`features.py`, `linear_probe.py`, `knn.py`), `src/diagnostics/representation.py` и `src/analysis/label_efficiency.py`. probe и диагностику представлений вы будете применять к кодировщикам следующих работ.

Как проверить себя. Тесты 4–6 проходят. Повторное вычисление признаков совпадает с первым с точностью 10^{-6}. Иначе кодировщик не в режиме `eval` или включена аугментация.

9 Эксперименты

Перед каждым экспериментом запишите гипотезу: что вы ожидаете и почему, со ссылкой на формулу раздела 5. E1 и E2 — поисковые, один seed на конфигурацию. Главный вывод проверяется в E3 на трёх seeds. Расхождение с гипотезой допустимо, если причина найдена и проверена. Не меняйте одновременно кодировщик, аугментации и функцию потерь, объясняя один эффект.

Общие условия: $N = 256$, τ из профиля. Оптимизатор и расписание скорости обучения одинаковы во всех запусках SimCLR, например SGD с импульсом и косинусным расписанием. Полное обучение длится $E_{\text{full}} = 100$ эпох, укороченное — $E_{\text{short}} = 25$. Точности измеряются на `test`, всё выбирается по `val`.

9.1 Эксперимент E1. SimCLR против базовых моделей

Вопрос. Насколько представление без меток лучше случайного кодировщика и насколько хуже обученного с учителем? Растёт ли качество вместе со снижением функции потерь?

Что сделать. Сравните три кодировщика одной архитектуры. B_0 — случайный, без обучения. B_1 обучен с учителем на всех метках `train` за E_{short} эпох с обрезкой и отражением. Выходной слой отбрасывается. SimCLR обучен с базовым набором из профиля за E_{full} эпох.

1. После каждой эпохи SimCLR записывайте функцию потерь, а на диагностической выборке — сходства пар, $\operatorname{tr} C$ и r_{eff} для h и $\hat{z}$, $\|\bar{\hat{z}}\|$. Оцените probeом не менее пяти контрольных точек, включая начальную.
2. Для B_0, B_1 и итогового SimCLR получите probe, kNN и сводку задания 3.
3. Постройте графики: функция потерь с линией $\ln(2N-1)$ и точность probe по эпохам; сходства пар; r_{eff} и $\operatorname{tr} C$ по эпохам; спектр C в логарифмическом масштабе в начале, середине и конце обучения.

Порог приёмки. Для трёх кодировщиков приведены точности probe и kNN на `test`, полученные одной процедурой. probe SimCLR измерен не менее чем в пяти точках, включая начальную. В конце обучения функция потерь не выше $\ln(2N-1)$. Если она

выше, ищите ошибку тестами 1–3. Знак $G(1)$ по (4.12) сообщается. Отрицательный знак допустим, если причина найдена.

Граница применимости. Есть ли этап, на котором функция потерь ещё снижается, а точность probe почти не растёт? Укажите такой этап и объясните его. Потери считаются по $\hat{z}$, а качество — по h. Достижимые значения потерь зависят от N и τ, см. раздел 5.3.

9.2 Эксперимент E2. Вклад аугментаций

Вопрос. Какие операции базового набора нужны для полезного представления CIFAR-10, какие почти не влияют, а какие мешают?

Что сделать. Базовый набор $\mathcal{T}$: обрезка с масштабированием, изменение цвета и третья операция из профиля.

1. Обучите SimCLR с набором $\mathcal{T}$ один раз, seed 1, за E_{short} эпох.
2. Для каждой операции o_j обучите SimCLR с набором $\mathcal{T}_{-j}$ при том же seed и тех же условиях.
3. Для каждого запуска постройте сводку задания 3. Вычислите вклад операции $\Delta_j = M(\mathcal{T}) - M(\mathcal{T}_{-j})$ по точности probe.
4. Постройте таблицу вариантов и столбчатую диаграмму Δ_j для probe и kNN.

Порог приёмки. Варианты различаются только аугментациями, что подтверждает `diff_runs` из ЛР0. Пусть $G_0 = M(\mathcal{T}) - M(\mathrm{B}_0)$ — выигрыш полного набора над случайным кодировщиком из E1 по probey. Операция критична, если $G_0 > 0$ и $\Delta_j \geq G_0/2$. Она критична и при коллапсе: `collapse_scale = true`, $\|\bar{\hat{z}}\| \geq 0{,}9$ или $r_{\text{eff}}/d < 0{,}2$. Операция мешает, если коллапса нет, $G_0 > 0$ и $\Delta_j \leq -G_0/2$. Иначе на одном запуске эффект неотличим от шума; так и запишите. Для каждой операции названа задаваемая ею инвариантность. Если $G_0 \leq 0$, правило половины выигрыша не применяется. Сообщаются знак Δ_j и наличие коллапса.

Граница применимости. Оставьте одну операцию — самую важную по Δ_j. Затем уберите все, чтобы виды совпали. Где качество резко падает, например до уровня B_0? Есть ли коллапс: $\|\bar{\hat{z}}\|$ близко к 1, $\operatorname{tr} C$ близко к 0 или $r_{\text{eff}}/d < 0{,}2$? Объясните по формуле (4.6), что происходит с притяжением при совпадающих видах и что делает отталкивание. Отсутствие коллапса — допустимый результат, если он объяснён.

9.3 Эксперимент E3. Основное сравнение при малой разметке

Вопрос. Сохраняется ли преимущество SimCLR над случайным кодировщиком при малой разметке? Выдерживает ли вывод смену размеченных изображений?

Что сделать. Новое предобучение не нужно: используются сохранённые признаки B_0, B_1 и SimCLR из E1.

1. До запусков запишите в файл `hypothesis.md` и зафиксируйте в Git: A — SimCLR, B — B_0, различие только в кодировщике; что зафиксировано ($r_{\min}$, подвыборки, probe, данные); ожидаемый знак и величину $\bar{\Delta}$; источник случайности — выбор

подвыборки и начальные веса probe; единицу анализа — изображение; какой результат опровергнет гипотезу.

2. Для seeds $s = 1, 2, 3$ и всех долей r из профиля обучите probe каждого кодировщика на одной и той же подвыборке $S_s(r)$. Точность измеряйте на `test`. При $r = 1$ подвыборка — весь `train`.
3. Вычислите парные разности (4.13) при $r = r_{\min}$.
4. Постройте кривые $M(r)$ трёх кодировщиков с логарифмической осью r, все seeds — точками, и кривую $G(r)$. Кривая B_1 — верхний ориентир: этот кодировщик видел все метки `train`.

Порог приёмки. Вывод «SimCLR лучше B_0 при $r_{\min}$» считается установленным, если все три разности Δ_s одного знака и $|\bar{\Delta}| \geq 2s_\Delta$. Иначе вывод формулируется как «различие не установлено». В отчёте приводятся все три Δ_s. Расхождение с гипотезой не снижает оценку, если оно объяснено.

Граница применимости. Seeds меняют выбор размеченных изображений и probe, но не предобучение: кодировщик SimCLR один. Поэтому вывод относится к этому кодировщику, а не к методу SimCLR в целом. Кроме того, регуляризация выбирается по 5 000 размеченным изображениям `val`, хотя в `train` меток меньше. Это одинаково для всех кодировщиков, но завышает качество при малой разметке. Укажите оба ограничения.

10 Контрфактические вопросы

Ответ опирается на ваши числа со ссылками на `RUN_ID`. Если для ответа нужен дополнительный запуск, выполните его.

1. Сравните функцию потерь E1 на первом и последнем шаге с уровнем $\ln(2N - 1)$. Почему потери запусков с разными N или τ нельзя сравнивать напрямую? Проверьте коротким запуском с $N = 64$.
2. Что изменится, если класс определяется цветом? Постройте контрольный набор, где класс задан оттенком, по «Протоколу данных курса» (раздел 4.6). Обучите probe на признаках двух кодировщиков E2: с изменением цвета и без него. Какой лучше различает такие классы?
3. Найдите в E1–E2 два запуска с близким r_{eff} представления h, но разной точностью probe, или наоборот. Можно ли судить о полезности представления по одному r_{eff}? При каком $\operatorname{tr} C$ вопрос теряет смысл?
4. Обучите probe на $\hat{z}$ последней контрольной точки SimCLR из E1 и сравните с h. Если $\hat{z}$ хуже, значит ли это, что голова вредна? Какие сведения она отбрасывает?
5. Сравните $G(r_{\min})$ и $G(1)$ из E3: где выигрыш больше? Почему разность $M_{\mathrm{B}_1}(r) - M_{\mathrm{SSL}}(r)$ при малой r нельзя считать ценой отказа от меток? Как изменить B_1, чтобы сравнение стало честным? Проверьте одним запуском.

11 Обязательные тесты

1. Два вида имеют ожидаемую форму. При выключенной аугментации они совпадают, при включённой — различаются, в том числе при `num_workers` > 0.

2. Функция потерь (4.5) с допуском 10^{-5} равна $\ln(2N-1)$ для одинаковых векторов и $\ln\bigl(1+(2N-2)e^{-1/\tau}\bigr)$, если пары совпадают, а разные пары перпендикулярны. При $\tau = 0{,}01$ и сходствах, равных 1, она конечна.
3. Градиент слагаемого ℓ_i по уже нормированному $\hat{z}_i$ от `autograd` совпадает с (4.6) с точностью 10^{-5}.
4. probe не меняет кодировщик: полный `state_dict`, включая буферы BatchNorm, совпадает до и после обучения. Соседи в kNN берутся только из `train`.
5. Для матрицы ранга 1 $r_{\text{eff}} = 1$, для $n = 20d$ независимых нормальных векторов $r_{\text{eff}} \geq 0{,}9\,d$. Для одинаковых векторов возвращается `n/a` и `collapse_scale = true` без исключения. Тождество (4.9) выполняется с точностью 10^{-5}.
6. Подвыборки меток стратифицированы и вложены: $S_s(r_1) \subset S_s(r_2)$ при $r_1 < r_2$. Запрос части `cal` вызывает ошибку.

12 Что сдать

Сдаются модули заданий 1–3, конфигурации E1–E3, файл `hypothesis.md`, тесты и отчёт. Отчёт состоит из шести частей:

1. **Профиль и протокол:** τ, третья операция, доли разметки, N, число эпох, оптимизатор, соблюдение правил для данных.
2. **E1:** таблица probe и kNN для B_0, B_1 и SimCLR, графики по эпохам, вывод о связи потерь и качества.
3. **E2:** таблица вариантов, Δ_j, группы операций, их инвариантности и граница применимости.
4. **E3:** кривые $M(r)$ и $G(r)$, гипотеза, все Δ_s, вывод.
5. **Ответы на контрфактические вопросы.**
6. **Ограничения:** что покрывают seeds, выбор по `val`, укороченное обучение в E2.

Каждое число сопровождается ссылкой на `RUN_ID`; графики строятся автоматически из сохранённых результатов.

13 Критерии завершения и оценивания

Работа завершена, если одновременно:

- эксперименты E1–E3 выполнены, у каждого результата есть `RUN_ID`;
- тесты проходят;
- изображения `val` и `test` не участвовали в предобучении, часть `cal` не читалась;
- файл `hypothesis.md` зафиксирован в Git до запусков E3;
- для каждого кодировщика есть сводка задания 3;
- на все контрфактические вопросы даны ответы с собственными числами.

Веса частей оценки те же, что в ЛР0: реализация 0,15, экспериментальный протокол 0,30, диагностика 0,30, перенос 0,25.

14 Дополнительные задания

1. **Температура.** Обучите SimCLR за E_{short} эпох при τ, равной 0,05; 0,1; 0,2; 0,5 и 1,0. Измерьте потери, r_{eff}, точность probe, энтропию весов p_{ik} и наибольший вес. Запуск со скоростью обучения SGD, умноженной на τ, отделит перераспределение весов от множителя $1/\tau$ [13].
2. **Стресс-тест коллапса.** Уберите отрицательные примеры: $\ell_i^+ = -s_{i,j(i)}$. Минимум такой функции достигается и одним вектором для всех изображений; сиамская сеть без специальных мер к нему приходит [20]. Какая из величин $\|\bar{\tilde{z}}\|$, $\operatorname{tr} C$, `collapse_scale`, r_{eff} замечает коллапс первой и насколько раньше probe?
3. **Перенос на кадры лиц.** Кадры с метками эмоций по умолчанию из RAVDESS [21]; разбиение по актёрам, единица анализа — клип. Обучите SimCLR на кадрах `train`, беря в batch не больше одного кадра клипа: иначе кадры клипа станут ложными отрицательными примерами. Сравните случайный кодировщик, обученный на ImageNet (веса torchvision [4]) и SimCLR на лицах по macro-F1 probe и дообучения всей сети (fine-tuning). Учтите, что данные их предобучения различаются по объёму и источнику.

15 Контрольные вопросы

1. Что такое положительная пара и сколько отрицательных примеров у вида?
2. Что произойдёт, если обучать без отрицательных примеров?
3. Зачем нужна проекционная голова и почему качество оценивают по h?
4. Как температура влияет на веса p_{ik} и на масштаб градиента?
5. Чем полный коллапс отличается от коллапса по размерностям?
6. Почему r_{eff} не определён при $\operatorname{tr} C = 0$ и не приводится без $\operatorname{tr} C$?
7. Почему для probe кодировщик переводят в режим `eval`? Что добавляет kNN к probey?
8. Как аугментация задаёт инвариантность и всегда ли сильная аугментация полезна?

16 Итог

После ЛР4 вы умеете обучать представление без меток и проверять его несколькими способами. Набор аугментаций определяет, к каким изменениям изображения представление станет нечувствительным. Полезность представления показывают probe, kNN и диагностика коллапса, а не функция потерь.

Низкий pretraining loss не равен хорошему представлению. Инвариантность полезна только тогда, когда она сохраняет информацию, релевантную задаче.

Литература

[1] Николенко С., Кадурин А., Архангельская Е. Глубокое обучение. Погружение в мир нейронных сетей. — СПб.: Питер, 2018. — 480 с.

[2] Гудфеллоу Я., Бенджио И., Курвилль А. Глубокое обучение / пер. с англ. А. А. Слинкина. — 2-е изд., испр. — М.: ДМК Пресс, 2018. — 652 с.

[3] PyTorch Documentation: Transforming and augmenting images (torchvision.transforms). https://docs.pytorch.org/vision/stable/transforms.html

[4] PyTorch Documentation: Models and pre-trained weights (torchvision.models). https://docs.pytorch.org/vision/stable/models.html

[5] PyTorch Documentation: torch.utils.data. https://docs.pytorch.org/docs/stable/data.html

[6] PyTorch Documentation: torch.nn.Module. https://docs.pytorch.org/docs/stable/generated/torch.nn.Module.html

[7] Hugging Face Community Computer Vision Course: Losses. https://huggingface.co/learn/computer-vision-course/en/unit4/multimodal-models/clip-and-relatives/losses

[8] T. Chen, S. Kornblith, M. Norouzi, G. Hinton. A Simple Framework for Contrastive Learning of Visual Representations. *ICML*, 2020.

[9] Y. Tian, C. Sun, B. Poole, D. Krishnan, C. Schmid, P. Isola. What Makes for Good Views for Contrastive Learning? *NeurIPS*, 2020.

[10] T. Xiao, X. Wang, A. A. Efros, T. Darrell. What Should Not Be Contrastive in Contrastive Learning. *ICLR*, 2021.

[11] K. He, X. Zhang, S. Ren, J. Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016.

[12] A. van den Oord, Y. Li, O. Vinyals. Representation Learning with Contrastive Predictive Coding. arXiv:1807.03748, 2018.

[13] F. Wang, H. Liu. Understanding the Behaviour of Contrastive Loss. *CVPR*, 2021.

[14] T. Wang, P. Isola. Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere. *ICML*, 2020.

[15] L. Jing, P. Vincent, Y. LeCun, Y. Tian. Understanding Dimensional Collapse in Contrastive Self-Supervised Learning. *ICLR*, 2022.

[16] O. Roy, M. Vetterli. The Effective Rank: A Measure of Effective Dimensionality. *EUSIPCO*, 2007.

[17] Q. Garrido, R. Balestriero, L. Najman, Y. LeCun. RankMe: Assessing the Downstream Performance of Pretrained Self-Supervised Representations by Their Rank. *ICML*, 2023.

[18] Z. Wu, Y. Xiong, S. X. Yu, D. Lin. Unsupervised Feature Learning via Non-Parametric Instance Discrimination. *CVPR*, 2018.

[19] A. Krizhevsky. Learning Multiple Layers of Features from Tiny Images. Technical report, University of Toronto, 2009.

[20] X. Chen, K. He. Exploring Simple Siamese Representation Learning. *CVPR*, 2021.

[21] S. R. Livingstone, F. A. Russo. The Ryerson Audio-Visual Database of Emotional Speech and Song (RAVDESS). *PLoS ONE*, 13(5), 2018.